Improve Network Performance with Topology Aware Routing in Kubernetes

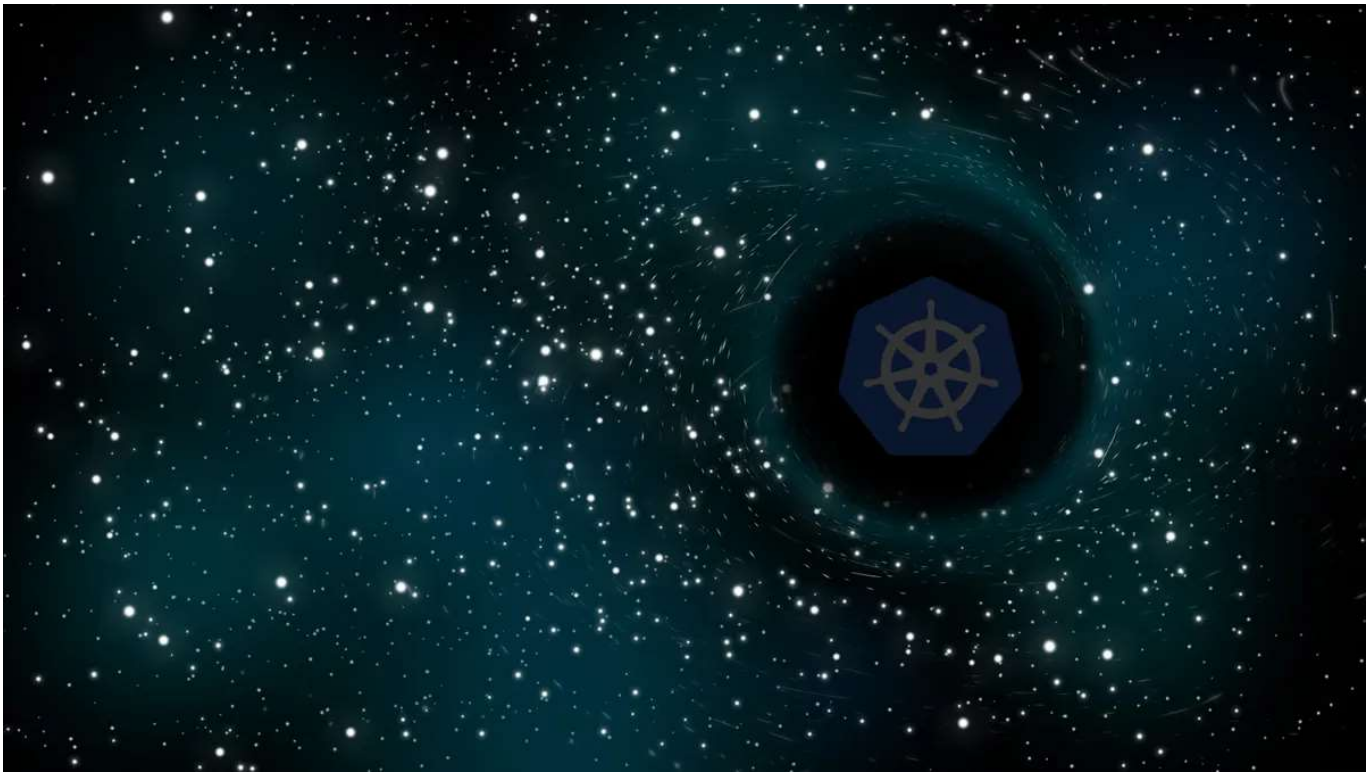


Seifeddine Rajhi · Follow

7 min read · Jan 30, 2024

130

How Topology Aware Routing Can Reduce Costs and Improve Network Performance 🚀



🚀 Introduction:

As Kubernetes clusters are increasingly deployed in multi-zone environments, it becomes important to keep network traffic within the zone where it originated.

Topology Aware Routing is a feature in Kubernetes that adjusts routing behavior to prefer keeping traffic in the zone it originated from.

This can help reduce costs or improve network performance by minimizing inter-zone traffic and ensuring that traffic reaches its destination with minimal delay.

In this blog post, we will explore the concept of Kubernetes topology aware routing and discuss its benefits in terms of reducing latency and optimizing network traffic flow.

We will also show you how to enable Topology Aware Routing for a Service and minimize the performance impact of overlay networking.

> *In this blog post, we will refer to Topology Aware Routing as* ***TAR****.*

Table of Contents:

- ***Impact of TAR on Network Traffic in Amazon EKS***
- ***Services in Kubernetes***
- ***How TAR Works***
- ***Hands-on Example***
  - *Prerequisites*
  - *Deploy cluster*
  - *Deploy the demo application*
  - Run initial load testing
  - Enable TAR and re-run load testing

Impact of TAR on Network Traffic in Amazon EKS:

A best practice for architecting resilient systems in AWS is to leverage multiple Availability Zones (AZs).

AWS Regions are composed of multiple AZs designed to be independent of each other and are separate by a meaningful physical distance to avoid correlated failure scenarios.

AWS EKS makes it possible for customers to run their mission-critical workloads across multiple distinct AZs, providing increased availability by combining Amazon's global infrastructure with Kubernetes constructs such as pod topology spread constraints.

However, when architecting workloads in this manner, there are considerations in terms of both latency and cost to consider. Pods within the

same Amazon EKS cluster can easily communicate using a Kubernetes Cluster IP Service, which routes traffic to its associated pods even if they are in different AZs.

> *Although this is a convenient way to take advantage of multiple AZs, some customers may want to bias traffic routing to pods in the same AZ as the originator. Keeping traffic in the same AZ benefits latency-sensitive workloads for*

Open in app ↗ Sign up Sign in

Medium Search Write

Kubernetes version 1.23 and became available in Amazon EKS version 1.24.

It's intended to provide a mechanism that attempts to keep traffic closer to its origin within the same AZ or in another location.

Services in Kubernetes:

Before we introduce how TAR works, let's introduce how a Kubernetes Service works.

When you create a Service, you specify a selector to target specific pods. Kubernetes creates an ***Endpoints object***, which it keeps populated with the IP addresses of the relevant pods.

Next, the Kubernetes controller allocates a Cluster IP to the Service, which is a virtual IP and only routable in worker nodes of the Kubernetes cluster. The instance of ***kube-proxy*** running on each node via a DaemonSet creates a series of ***iptables rules*** (or IP Virtual Server (IPVS) virtual servers, if enabled). When a traffic is targeted Cluster IP (or enter the cluster via NodePort) either

iptables or IPVS routes the packet to a pod based on their load balancing algorithm.

By default, kube-proxy adds all pods in an Endpoints object to each node's iptables or IPVS without taking into account aspects, such as the AZs the pods are running in or which zone the traffic is originating from.

This means that packets are routed indiscriminately between AZs, which potentially introduces latency and incurring inter-AZ data traffic costs.

How TAR Works:

TAR enables an alternative mechanism for traffic routing by putting a subset of endpoints to kube-proxy. TAR depends on ***EndpointSlices*** controller, which is a more extensible alternative to Endpoints. Endpoints on Endpoints controller only contain pod IP address and ports, while in EndpointSlices it can be set with hints. Hints are additional labels on each endpoint.

TAR sets a hint of which zone the endpoint resides. When a hint is set, kube-proxy filters the endpoints based on the hints. In most cases, it chooses endpoints in the same zone. When traffic enters kube-proxy, it's routed only to pods in the same zone which is free of traffic costs.

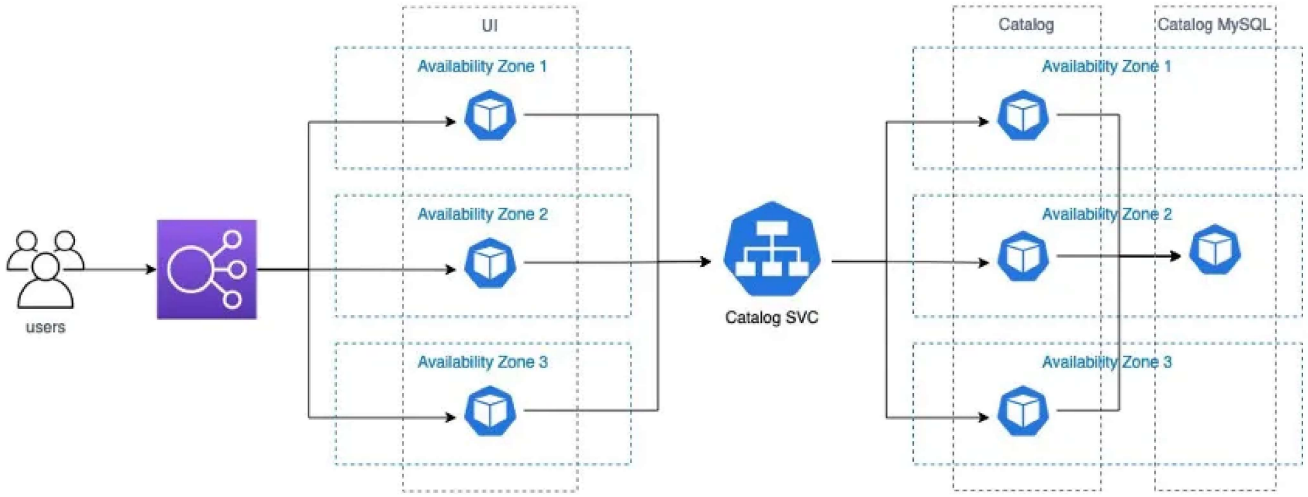
Hands-on Example:

In this walkthrough, you'll deploy an Amazon EKS cluster usinbg Terraform.

You can design your infrastructures as templates and use Terraform to provision, manage, and destroy them.

Besides the cluster itself, Terraform also deploys the AWS Distro for OpenTelemetry (ADOT) Operator via its Amazon EKS Blueprints add-on. The ADOT Operator enables a simplified experience for instrumenting your applications running on Amazon EKS to capture metrics and tracing data. We'll use the ADOT Operator and AWS X-Ray to visualize the requests between workloads running on the Amazon EKS cluster.

After the Amazon EKS cluster and its related infrastructure are created, we deploy the containers sample application, which shows traffic between separate front-end and back-end components. This allows us to explore how different configurations of Amazon EKS worker nodes and pods affects TAR.



architecture diagram

Prerequisites:

For this demo, you need the following:

- Basic understanding of Linux operating systems and Kubernetes.
- An AWS
- Administrator or equivalent access to deploy the required resources.

- AWS Command Line Interface (AWS CLI) installed and configured.
- Terraform CLI installed.
- Git CLI installed.
- kubectl, and helm client installed.

Deploy cluster:

We prepared a Terraform template to deploy Amazon EKS cluster and ADOT Operator. Start by cloning the sample source code repository from GitHub.

```
git clone https://github.com/seifrajhi/topology-aware-routing-tf
cd topology-aware-routing-tf/terraform
```

Run the following command to deploy the cluster.

```
export AWS_REGION="eu-west-1"
terraform init
terraform plan
terraform apply -auto-approve
```

Once the Terraform has completed, then you can set up kubectl by running this command:

```
aws eks --region $AWS_REGION update-kubeconfig --name tar-demo-cluster
```

Deploy the demo application:

Run the following command to deploy the sample application:

```
cd ../kubernetes
kubectl apply --server-side -f common.yaml
kubectl apply --server-side -f simple.yaml
```

Get the endpoint for UI component by running the following command:

```
kubectl get svc ui-lb -n ui -o jsonpath={.status.loadBalancer.ingress[0].hostnam
```

You can access the demo application by accessing this link. This displays an example online shopping site.

Run initial load testing:

Now we have deployed the Amazon EKS cluster and demo application, we can run a load test to simulate traffic and see how traffic between front-end and back-end is routed. We'll use hey to generate traffic, which is a tiny program that can send HTTP requests.

Before you test topology-aware routing, you may want to know how traffic been routed without TAR for comparison. You can determine this by performing a load test against this application, and traffic shows on AWS X-Ray service graph.

- Run load test against UI load balancer. Run the following command:

```
export UI_ENDPOINT=$(kubectl get svc ui-lb -n ui -o jsonpath='{.status.loadBalan
kubectl run load-generator \
  --image=williamyeh/hey:latest \
  --restart=Never -- -c 10 -q 5 -z 60m http://$UI_ENDPOINT/home
```

Enable TAR and re-run load testing:

Enabling Topology Aware Routing is easy: simply set the `service.kubernetes.io/topology-mode` annotation to `Auto` on a `Service`. The `EndpointSlice` controller will try to assign hints to each endpoint in an `EndpointSlice` to steer traffic to `Pods` within a given zone.

It's important to understand the safeguards and constraints implemented by the `EndpointSlice` controller and the kube-proxy, especially if routing isn't working as you expect.

> ***Note:*** *Prior to Kubernetes 1.27, this behavior was controlled using the* `service.kubernetes.io/topology-aware-hints` *annotation.*

Now let's enable TAR and re-run the load test to see if that improves traffic routing. Run the following command to enable TAR on the catalog Service:

```
kubectl -n catalog annotate svc catalog service.kubernetes.io/topology-mode=Auto
```

You can run the following command to check if hints are set:

```
kubectl get endpointslices -l kubernetes.io/service-name=catalog -n catalog -o y
```

You'll get results similar to the following example. If you see hints and a corresponding AZ name, then the TAR is enabled and was successfully activated.

```
addressType: IPv4
apiVersion: discovery.k8s.io/v1
endpoints:
- addresses:
  - 10.0.10.100
  conditions:
    ready: true
    serving: true
    terminating: false
  hints:
    forZones:
    - name: eu-west-1a
```

The control plane is very conservative about Topology Aware Routing decisions. If you run into issues, such as errors in the `Service` events or you find that hints are not populating on the `EndpointSlice` , then you should double-check that your topology labels have been applied and that `Nodes` are reporting values for allocatable CPU.

Re-run the load test with the following command:

```
export UI_ENDPOINT=$(kubectl get svc ui-lb -n ui -o jsonpath='{.status.loadBalan
kubectl run load-generator \
  --image=williamyeh/hey:latest \
  --restart=Never -- -c 10 -q 5 -z 60m http://$UI_ENDPOINT/home
```

Now you know that each UI pod only talks to one catalog pod. You can also find that UI pod always talks to catalog pod that resides on the same AZ. No inter-AZ data traffic cost is incurred in this scenario.

END End notes:

Topology Aware Routing is a useful feature in Kubernetes that helps manage traffic flow by keeping it within the same zone where it originated. While it may not be suitable for every workload, it can be beneficial in environments with evenly distributed zonal traffic.

In this blog post, we explored TAR and its effects on network traffic routing in Amazon EKS. We also provided a walkthrough on how to enable TAH for a Service and minimize the performance impact of overlay networking.

Until next time 🎉 *PS*



Photo by Tsuyuri Hara on Unsplash

Thank you for Reading !! 🙌🏻😁📃, see you in the next blog. 🤘 PS

🚀 Thank you for sticking up till the end. If you have any questions/feedback regarding this blog feel free to connect with me :

♻️ PS**LinkedIn**: https://www.linkedin.com/in/rajhi-saif/

♻️ PS **Twitter** : https://twitter.com/rajhisaifeddine

The end ✌🏻

🔰 Keep Learning !! Keep Sharing !! 🔰

Kubernetes | Platform Engineering | Sre | DevOps | Kubernetes Networking



Written by Seifeddine Rajhi

Follow

12.5K Followers

I build and break stuff, preferably in the cloud, ♥ OpenSource. Twitter: @rajhisaifeddine

More from Seifeddine Rajhi





 Seifeddine Rajhi

Most common mistakes to avoid when using Kubernetes: Anti-...

A simple guide to avoid these Pitfalls 🕳 🧍

10 min read · Jan 22, 2024

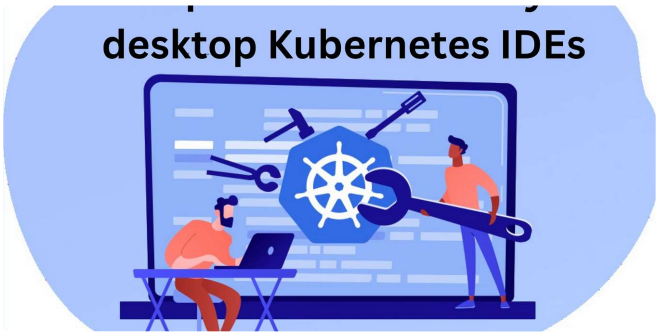
 Seifeddine Rajhi in ITNEXT

Understanding Kubernetes Probes: Better Apps Health checks 🚥

Guide to Liveness, Readiness, and Startup Probes ♻

8 min read · Jan 22, 2024

693 9 181



Seifeddine Rajhi

Explore user-friendly Desktop Kubernetes open-source IDEs

making Kubernetes cluster management accessible and simple to all

8 min read · Feb 8, 2024

248 3



Seifeddine Rajhi

Building Container Images in Kubernetes Cluster with Kaniko

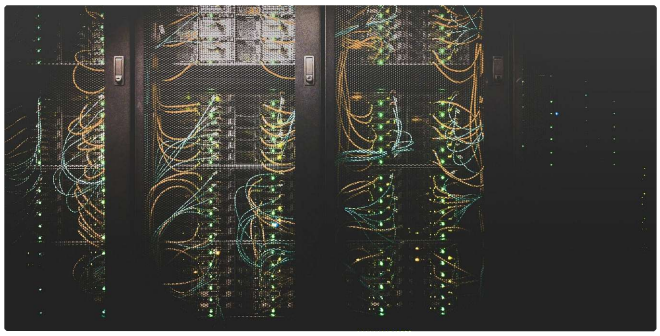
Daemonless Docker Image Building in k8s

7 min read · Jan 23, 2024

223 3

See all from Seifeddine Rajhi

Recommended from Medium





Jack Lindamood

(Almost) Every infrastructure decision I endorse or regret after ...

Assortment of technology start infrastructure rec

15 min read · Feb 6, 2024

504 9

Anders Jönsson

Lessons From Our 8 Years Of Kubernetes In Production — Two...

Cluster Crashes, Battling Complexity, Scaling, Power Of Helm, Tracing & Observability, Fro...

12 min read · Feb 5, 2024

925 13

Lists



General Coding Knowledge

20 stories · 931 saves



Natural Language Processing

1215 stories · 689 saves



Productivity

237 stories · 329 saves

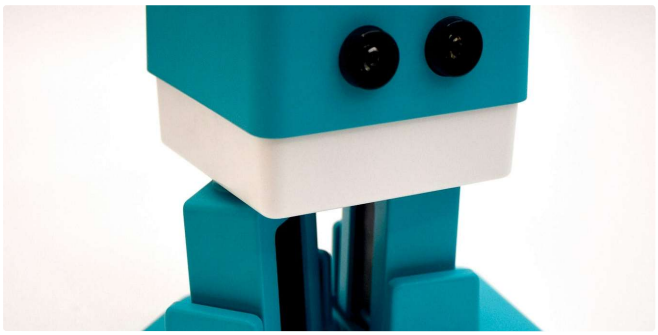


Eduardo Messuti in StatusPal

Best Programming Languages for DevOps in 2024 (With Examples)

6 min read · Feb 5, 2024

226 1



Akhilesh Mishra

You should stop writing Dockerfiles today — Do this instead

Using docker init to write Dockerfile and docker-compose configs

5 min read · Feb 8, 2024

2.4K 23



Madokai

This is Why I Didn't Accept You as a Senior DevOps Engineer

Ex-Principal DevOps argues today's "senior" engineers lack true leadership, despite titles...

· 10 min read · Jan 29, 2024

190 1



Javier Canizalez in Terrakube

Introducing Terrakube 2.19.0 🚀

OpenTofu Now Available on Terrakube

4 min read · Feb 2, 2024

20 1

See more recommendations